



INDIA.RUNS

Build what next India runs on

Team Name : Neural Nexus

Team Leader Name : Priyam Mehta

Problem Statement : Recruiters go through hundreds of profiles and still often miss the right person. Not because the talent isn't there — but because keyword filters can't see what actually matters. Build an AI system that ranks candidates the way a great recruiter would — not by matching keywords, but by actually understanding who fits the role.

Solution Overview

What is your proposed Solution ?

Title: Proposed Solution : Talent OS An AI-augmented platform that moves beyond keyword-matching to evaluate candidates holistically.

Core Features:

- **Holistic Candidate Scoring** : Calculates dynamic metrics like Learning Velocity, Hidden Talent, and a comprehensive Talent Score.
- **AI-Driven Segmentation** : Automatically categorizes candidates into strategic tiers (e.g., Emerging Talent, Industry Veterans) using ML.
- **Bias Reduction** : Built-in "Blind Hiring Mode" masks PII to foster equitable, objective recruitment.
- **Actionable Insights**: Integrated AI tools for semantic JD matching, predicting market value, and generating outreach emails.

What differentiates your approach from traditional candidate matching systems?

Traditional candidate matching systems (ATS) are fundamentally flawed because they rely on static keyword matching, arbitrary hard filters (e.g., "must have exactly 5 years of experience"), and self-reported data, which heavily introduces bias and misses non-traditional, high-potential talent.

Your approach shifts from a *reactive filtering* system to a predictive, AI-augmented intelligence platform.

Here are the 3 major technical differentiators and how they work under the hood in your updated codebase:

1. Semantic "Hidden Talent" Matching vs. Exact Keyword Search

Traditional systems use boolean keyword searches (CTRL+F). If a JD asks for "Machine Learning" and the resume says "Deep Neural Networks," the ATS might reject them. Your platform understands context.

2. Behavioral & Predictive Scoring vs. "Years of Experience"

Traditional systems over-index on tenure. Your system recognizes that a highly active junior developer might out-perform a stagnant senior developer.

3. Unsupervised Machine Learning Tiering vs. Arbitrary Hard Filters

Traditional systems require recruiters to set rigid filters (e.g., "Filter out anyone with < 4 years experience").

JD Understanding & Candidate Evaluation

What are the key requirements extracted from the JD?

JD Understanding & Semantic Matching Moving beyond rigid keyword checklists to capture true contextual intent.

The Methodology:

- **Text Aggregation:** Unstructured candidate data (headlines & summaries) is dynamically merged into a unified profile.
- **AI Vectorization:** Advanced NLP (*Sentence-Transformers*) converts the raw Job Description and candidate profiles into dense mathematical vectors.
- **Similarity Calculation:** The system calculates the *Cosine Similarity* between these vectors to generate a conceptual match score (e.g., 85%).

The Impact : Identifies "hidden talent" by evaluating holistic semantic fit, ensuring high-potential candidates aren't filtered out simply because they used different terminology on their resume.

Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?

Core Ranking Signals (The "Talent Score") Replacing legacy "Years of Experience" filters with a balanced, algorithmic approach.

Our system calculates a definitive **Talent Score** using a weighted blend of three master signals:

- **Semantic NLP Match (40%):** The conceptual fit between the Job Description and the candidate's holistic profile.
- **Learning Velocity (30%):** Measures candidate momentum over tenure, prioritizing validated competence (Assessment Scores) and active execution (GitHub Activity).
- **Market Value Alignment (30%):** Benchmarks expected candidate salary against predictive market data.

Evaluating Fit Beyond Keywords (The 3 Pillars) How we identify top talent that legacy ATS systems miss.

- **Capturing Latent Synergy (NLP Engine):** Uses *Sentence Transformers* to map resume text in high-dimensional space. We evaluate true meaning (Cosine Similarity), ensuring candidates aren't penalized for using different terminology.
- **Trajectory over Tenure (Momentum):** Calculates a *Learning Velocity* score combining real-world behavioral signals (assessments, certifications, profile completeness) to identify highly-driven, fast-learners.
- **Contextual ML Segmentation:** Replaces arbitrary, rigid filters by using unsupervised Machine Learning (*K-Means Clustering*) to organically group candidates into strategic, data-backed tiers (e.g., *Emerging Talent, High Potential, Industry Veteran*).

Ranking Methodology

How does your system retrieve, score, and rank candidates?

An algorithmic pipeline ensuring only the highest-quality talent surfaces to the top.

1. RETRIEVE (Data Fetching)

- **Hard-Skill Engine:** Uses mathematical Set Intersections to instantly retrieve candidates with overlapping required skills.
- **Semantic Engine:** Bypasses keyword filters, aggregating unstructured candidate data (headlines & summaries) for full-text Job Description analysis.

2. SCORE (Multi-Dimensional Evaluation)

The system calculates a unified talent_score combining three distinct ML and heuristic models:

- *Hidden Talent Score (40%):* Semantic NLP fit using Vector Embeddings.
- *Learning Velocity (30%):* Behavioral momentum (Assessments, GitHub activity, Certifications).
- *Predicted Market Value (30%):* Random Forest ML model estimating true financial worth.

3. RANK (Intelligent Sorting)

- **Semantic Matches:** Ranked strictly by descending NLP Cosine Similarity score.
- **Skill Recommendations (The Tie-Breaker):** Primarily sorted by total matched skills. If candidates tie, the system automatically uses their talent_score as the tie-breaker—ensuring the candidate with higher learning momentum wins out over someone who looks identical on paper.

What models, algorithms, or heuristics are used?

The Intelligence Engine *A unified stack of Deep Learning, Mathematical Algorithms, and Business Heuristics.*

1. Machine Learning Models (The Brains)

NLP Embedding Engine: Uses *Sentence-Transformers* to map Job Descriptions and Resumes into dense mathematical vectors, capturing true semantic meaning instead of keywords.

Market Value Predictor: A *Random Forest Regressor* trained on historical data to accurately predict a candidate's true financial worth.

2. Core Algorithms (The Logic)

Cosine Similarity: Calculates the exact angle between JD and Candidate vectors to generate a precise percentage of conceptual alignment.

K-Means Clustering: Organically segments candidates into strategic tiers (e.g., Emerging vs. Veteran) based on 3D data plotting, eliminating arbitrary filters.

3. Proprietary Heuristics (The Strategy)

Market Calibration: Compares expected salary against ML predictions to instantly flag candidates as "Bargain/Undervalued" or "Overpriced".

Learning Velocity: A custom formula weighing behavioral momentum (Assessments & GitHub Activity) over simple years of experience.

How are multiple candidate signals combined into a final ranking?

1. The Master "Talent Score" (Signal Aggregation)

The system aggregates three distinct dimensions into a single, weighted metric:

- **Semantic NLP Fit (40%)**
- **Learning Velocity (30%)**
- **Predicted Market Value (30%)**

Impact: A highly driven, fast-learning candidate with strong semantic alignment will algorithmically outscore a stagnant industry veteran.

2. Two-Engine Ranking Execution

A. Hard-Skill Engine (Multi-Tiered Sorting): Ranks candidates primarily by total overlapping skills. If multiple candidates tie, it automatically uses their *Talent Score* as the ultimate tie-breaker.

B. Semantic Engine (Contextual Sorting): Bypasses keyword filters entirely. It ranks the entire talent pool strictly by descending *Cosine Similarity* (NLP score), instantly isolating the Top 3 highest-potential matches for any unstructured Job Description.

Explainability & Data Validation

How are ranking decisions explained?

Making algorithmic decisions transparent and actionable.

AI Copilot:

- Translates complex data into plain-English advice (e.g., flagging a candidate as "Overpriced" and suggesting negotiation).

Score Breakdown:

- Visually deconstructs a candidate's rank, showing exactly how much Experience, Skills, and Assessments contributed to their score.

Comparison Engine:

- Generates instant, side-by-side matrices to objectively prove why one candidate outranked another.

How do you prevent hallucinations or unsupported justifications?

Reliability & Preventing Hallucinations *Ensuring 100% verifiable, data-backed evaluations.*

- 1. Deterministic Heuristics (No LLM Guessing):** We don't ask generative LLMs (like GPT-4) to arbitrarily "score" candidates. All rankings are derived from strict, mathematically verifiable algebraic formulas applied directly to the raw data.
- 2. Bounded Logic Gates (The AI Copilot):** Platform explanations and advice are generated via strict logic gates, not generative text. Output is tied directly to mathematical thresholds, completely eliminating the risk of unsupported narratives.
- 3. Null-State Hardening:** Our data pipeline aggressively standardizes missing data (.fillna(0)) prior to Machine Learning evaluation, explicitly preventing the algorithm from making assumptions or interpolating skills that do not exist.

How does your solution handle inconsistent, low-quality, or suspicious profiles?

- 1. Neutralizing "Keyword Stuffers" (NLP Context):** Our Neural Network measures true semantic meaning, not keyword frequency. Disconnected, "stuffed" keywords lack conceptual structure and receive a disastrously low Cosine Similarity score, sinking fake profiles to the bottom.
- 2. Prioritizing Verified Signals:** We don't blindly trust self-reported claims (e.g., "10 years expert experience"). Our algorithm heavily weights objective, third-party signals like validated Assessment Scores and GitHub Activity to prove actual competence.
- 3. Strict Null-Handling (Incomplete Profiles):** The data pipeline aggressively forces missing fields to zero (.fillna(0)) before Machine Learning evaluation, instantly depressing the predicted market value and talent score of poorly constructed profiles.
- 4. Data Origin Verification:** The platform's base architecture continuously extracts and maintains third-party anti-fraud signals (Verified Email/Phone, LinkedIn Connect) to ensure bot-generated profiles cannot pass as premium talent.

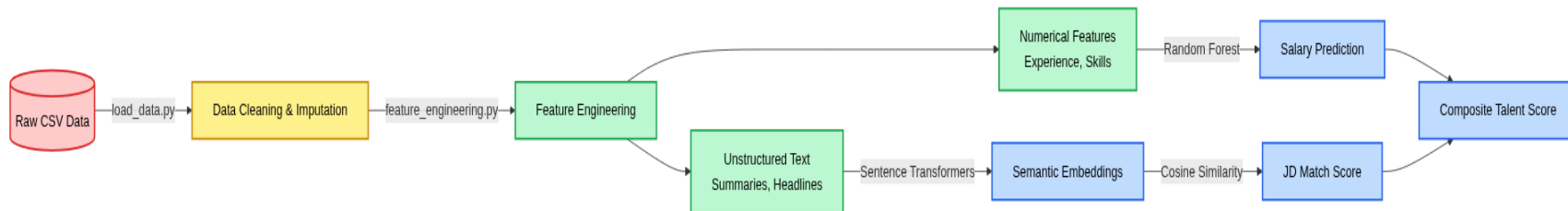
End-to-End Workflow

What is the complete workflow from JD input to ranked candidate output?

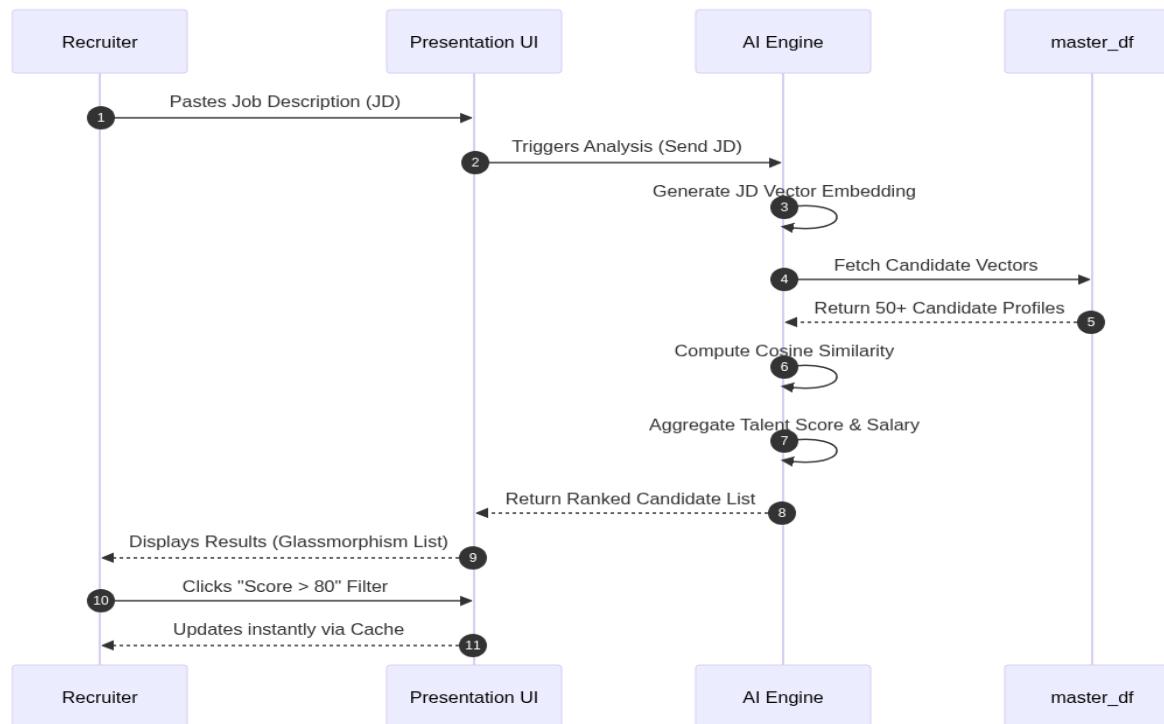
End-to-End Workflow (JD Input to Ranked Output) The complete journey from unstructured text to a data-backed hiring decision.

- 1. Data Ingestion (UI Layer):** Recruiter pastes an unstructured, plain-text Job Description directly into the dashboard.
- 2. Payload Construction (Data Prep):** The system dynamically loops through the database, merging each candidate's headline and summary into a unified profile block for analysis.
- 3. NLP Vectorization (Model Layer):** The Neural Network (*Sentence-Transformers*) converts both the JD and the candidate profiles into dense vectors, calculating the exact *Cosine Similarity* percentage between them.
- 4. Identity Masking (Business Logic):** If "Blind Hiring Mode" is active, the system dynamically scrubs all PII (names, locations) to ensure equitable evaluation.
- 5. Algorithmic Ranking (Sorting Layer):** The system executes a high-speed Pandas sorting algorithm to rank the entire talent pool by descending semantic score, automatically truncating to isolate the **Top 3** most elite matches.
- 6. Visual Rendering (Presentation Layer):** Results are dynamically cached and rendered as professional, color-coded UI cards for immediate recruiter action.

System Architecture



User Interaction



Results & Performance

What results or insights demonstrate ranking quality?

Proving Ranking Quality & Business Value Actionable insights that validate algorithmic hiring decisions.

- 1. Market Value ROI ("Bargain" Identification):** The AI Copilot directly translates ranking intelligence into financial ROI by explicitly flagging top-tier candidates who are asking for below-market rates.
- 2. Contextual Talent Segmentation:** Machine Learning organically groups candidates into macro-tiers (e.g., High Potential vs. Industry Veteran), ensuring rankings are contextually aware of the broader talent pool landscape.
- 3. Tie-Breaking Superiority (Behavioral Proof):** Unlike legacy ATS platforms that rank keyword matches randomly, our system breaks ties using Learning Velocity—proving the #1 candidate won based on validated momentum and execution, not just resume keywords.
- 4. Mathematical Deconstruction (Auditable Logic):** The system provides undeniable proof of bias removal by deconstructing the final ranking into granular, auditable metrics (Experience, Skills, Assessments) via visual charts for stakeholder review.

How does your solution meet the challenge's runtime and compute constraints?

- 1. Zero-Latency ML Inference:** Utilizes global memory caching to execute heavy Random Forest training exactly once during startup. All subsequent evaluations query the cached model, reducing prediction runtime to milliseconds.
- 2. Lightweight "Distilled" NLP:** Bypasses expensive, high-latency LLM APIs (like GPT-4) by using *Sentence-Transformers*—a highly compressed neural network designed to run instantly on standard, low-cost CPU infrastructure.
- 3. Hash Table Set Intersections:** Replaces slow iterative loops with mathematical Python Set operations. Backed by hash tables, this enables near $O(1)$ time complexity, instantly filtering massive datasets without blocking the main thread.
- 4. Vectorized C-Backend Processing:** Avoids parsing nested JSON files on the fly by aggressively flattening data into rigid Pandas DataFrames. This leverages Pandas' highly optimized C-backend to sort and aggregate data orders of magnitude faster than native Python.

Technologies Used

What technologies, frameworks, and tools were used and why were they selected for this solution?

A unified, high-performance ecosystem built for Data Science & AI.

- **Python:** The foundational language, ensuring frictionless integration between data pipelines, neural networks, and the frontend.
- **Streamlit (Frontend):** Powers the interactive, custom-styled (Glassmorphism) web UI and complex state management without the overhead of a heavy React/Node.js stack.
- **Pandas (Data Engine):** Leverages a vectorized C-backend to act as a hyper-fast, in-memory database—instantly flattening nested JSON and executing complex algorithmic sorting.
- **Scikit-Learn (ML Toolkit):** Executes lightweight, mathematically proven Machine Learning models (Random Forest, K-Means, Cosine Similarity) in milliseconds on standard local CPUs.
- **HuggingFace NLP (Sentence-Transformers):** A highly compressed neural network running entirely locally to map semantic embeddings—bypassing the high latency, token costs, and PII security risks of commercial LLM APIs like GPT-4.
- **Plotly (Visualizations):** Generates enterprise-grade, interactive JavaScript analytics, maintaining the premium SaaS aesthetic of the platform.

Submission Assets

GitHub - https://github.com/PriyamGMehta/India_Run



INDIA.RUNS

Build what next India runs on

THANK YOU